

RelGate: Evidence-Grounded Readiness Gates for LLM-Reviewed Cloud Changes

Lokesh Chauhan
Independent Researcher
Dallas, TX, USA
lokesh0186@gmail.com

Abstract—Pre-deployment review of cloud changes is a reliability activity: reviewers must decide whether a change has enough evidence for observability, alerting, rollout safety, rollback, ownership, reliability impact, and validation. Large language models can assist this review, but free-form answers may be poorly calibrated. They can either approve unsupported changes or block changes that already contain sufficient readiness evidence. We present RelGate, a project prototype and experimental framework that requires LLM-generated readiness decisions to cite exact evidence from the change bundle or mark the evidence as missing. In a 12-scenario pilot across three models and three review modes (108 calls), all modes avoided false-ready approvals on unsafe changes, but free-form review blocked all READY controls. RelGate achieved the best calibration in this pilot, with zero observed false-ready decisions, zero observed false-blocks, and fewer unsupported evidence claims than checklist-only review (0.053 vs. 0.208). These early results suggest that evidence-grounded review may improve the auditability and release precision of LLM-assisted production-readiness gates.

I. MOTIVATION

Cloud services deploy many changes daily, each requiring production-readiness evidence for safe release [1]. Consider a scenario in which a change bundle includes explicit dashboard links, alert rules, a staged rollout plan, a rollback trigger, on-call ownership, SLO-impact analysis, and validation test results. In our pilot, free-form LLM review returned FIX-BEFORE-SHIP for such a fully-specified change across all three tested models, typically requesting generic additional assurance despite all evidence being present. Conversely, giving the same models a structured checklist produced correct READY decisions in most cases, but sometimes marked gates as satisfied without citing supporting text from the bundle.

The pilot exposes a calibration problem: free-form review can be safe but over-conservative, while checklist review can improve decisions yet encourage unsupported completion. More broadly, unsupported readiness claims create a false sense of auditability, where the gate says “pass” but the supporting evidence is fabricated or paraphrased rather than cited [2]. As LLM-assisted review enters production workflows [3], this calibration gap becomes a reliability concern. RelGate addresses it by requiring exact evidence citations for every passing gate, treating readiness review as an auditable

*This work was conducted independently and does not relate to the author’s employment. No employer resources, data, or confidential information were used.

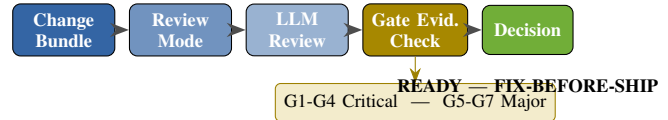


Fig. 1. RelGate workflow. The model evaluates a change bundle against seven readiness gates and must cite exact evidence for each passing gate; missing evidence produces a FIX-BEFORE-SHIP decision.

evidence-checking problem rather than a free-form advisory task.

II. RELGATE DESIGN

RelGate evaluates cloud changes against seven production-readiness gates: **G1** Observability (dashboard/metric/log evidence), **G2** Alerting (failure-mode coverage), **G3** Rollout Safety (staged deployment or blast-radius control), **G4** Rollback (revert plan with trigger), **G5** Ownership (on-call and escalation), **G6** Reliability Impact (SLO or error-budget analysis), and **G7** Validation (test/staging evidence). Gates G1-G4 are critical: if any critical gate lacks cited evidence, the decision must be FIX-BEFORE-SHIP. Gates G5-G7 are major: missing evidence triggers a warning but does not block.

The *evidence-grounding rule* requires the model to cite exact text from the change bundle for each PASS verdict. If no supporting text exists, the model must report MISSING_EVIDENCE and recommend a specific remediation. The model must not infer or fabricate evidence. This changes the review task from checklist completion to evidence auditing.

III. PILOT STUDY

We constructed 12 change scenarios: 9 with seeded reliability gaps (missing alerts, absent rollback plans, vague ownership) and 3 READY controls with complete evidence for all seven gates. Each scenario was evaluated under three prompting modes: **M0 Freeform** (unstructured review request), **M1 Checklist** (G1-G7 listed; pass/fail per gate), and **M2 Evidence-Grounded** (G1-G7 plus the evidence-grounding rule). Three models were used via OpenRouter at temperature 0: GPT-5.5, Grok 4.3, and Llama 4 Maverick. This yields $12 \times 3 \times 3 = 108$ evaluation calls. The artifact contains the scenario schema, prompts, raw outputs, scoring script, and summary metrics.

Metrics. We report Critical Gap Recall (fraction of seeded critical gaps identified), False-Ready Rate (unsafe changes

TABLE I
PILOT RESULTS ACROSS REVIEW MODES (AVERAGED OVER 3 MODELS, 12 CASES EACH).

Metric	M0	M1	M2
Critical Gap Recall	0.975	1.000	1.000
False-Ready Rate ($n=27$)	0.000	0.000	0.000
False-Block Rate ($n=9$)	1.000	0.111	0.000
Unsupported Evid. Claims	0.062	0.208	0.053
Decision Accuracy ($n=36$)	0.750	0.972	1.000
Actionability (0-2)	1.58	1.74	1.74

approved, $n=27$ per mode), False-Block Rate (safe controls blocked, $n=9$ per mode), Unsupported Evidence Claims (the fraction of explicit readiness-evidence claims not directly supported by the scenario text), Decision Accuracy ($n=36$ per mode), and Actionability (0-2 specificity scale).

IV. PRELIMINARY RESULTS

Table I summarizes results. We observe three findings:

(1) Free-form review is safe but over-conservative. M0 detected nearly all seeded gaps (0.975 critical recall) but blocked every change, including all READY controls (false-block = 1.000), yielding 0.750 decision accuracy.

(2) Checklist structure improved specificity but increased unsupported claims. M1 reduced false-block to 0.111 and reached 0.972 accuracy. However, its unsupported-evidence rate (0.208) was about four times higher than M2. The checklist structure encouraged models to mark gates as satisfied by paraphrasing text rather than citing verbatim evidence.

(3) Evidence grounding improved calibration by allowing READY controls while preserving zero observed false-ready approvals. M2 achieved 1.000 decision accuracy with no false-ready and no false-block. The evidence-grounding rule forced models to cite real bundle text or honestly report missing evidence, yielding the lowest unsupported-claim rate (0.053).

The main lesson is that production-readiness review should be evaluated as an evidence-auditing task, not only as a pass/fail classification task.

Qualitative example. On READY control C10 (a Kubernetes HPA addition with full evidence), all three free-form reviews returned FIX-BEFORE-SHIP despite complete readiness evidence. RelGate approved the same case by citing the rollout plan, rollback trigger, alert rule, dashboard, owner, SLO statement, and load-test results. In one M1 review of C08 (a queue-worker change with empty alerting), Grok 4.3 quoted a paraphrased rollback plan as evidence, illustrating unsupported completion.

V. DISCUSSION AND LIMITATIONS

Relevance to ISSRE. RelGate is intended as a discussion-starting project artifact: a small benchmark, prompt family, and scoring rubric for studying calibrated LLM-assisted change review. Production-readiness review is a reliability activity where missed gaps cause incidents and excessive blocking

delays safe releases [1], [4]. As cloud services increasingly adopt LLM-assisted workflows [3], calibration and auditability of automated gates become reliability concerns that ISSRE is well-positioned to address.

Limitations. This pilot uses 12 synthetic scenarios; real production changes exhibit greater diversity and ambiguity. READY controls are fully specified; real readiness evidence may be ambiguous, stale, or organization-specific. Evidence presence does not guarantee actual production safety; it only makes the decision auditable against the provided bundle. Model versions evolve rapidly; results with GPT-5.5, Grok 4.3, and Llama 4 Maverick may not generalize. We have not yet measured latency, cost, or developer trust in deployment.

Next steps. We plan to expand to 50+ scenarios from anonymized change records, conduct a developer study, and integrate RelGate into a CI/CD pipeline [5].

Artifact: <https://github.com/lokesh0186/relgate>

REFERENCES

- [1] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- [2] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *arXiv preprint arXiv:2311.05232*, 2023.
- [3] H. Yan, Y. Chen, M. Ma, M. Wen, S. Lu, S. Zhang, T. Xu, R. Wang, C. Bansal, S. Rajmohan, C. Zhang, Q. Lin, and D. Zhang, "An empirical study of production incidents in generative AI cloud services," in *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2025.
- [4] H. S. Gunawi, M. Hao, R. O. Suminto, A. Laksono, A. D. Satria, J. Adityatama, and K. J. Eliber, "Why does the cloud stop computing? Lessons from hundreds of service outages," in *Proc. ACM Symp. Cloud Computing (SoCC)*, 2016.
- [5] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2023.